

# **TASK REPORT**

## **MITIGATION DDOS / EXCESSIVE-TRAFFIC DETECTION AND SECURE FILE-UPLOAD SYSTEM**

P.G. Senapathy Centre for Computing Resources  
Indian Institute of Technology Madras (IIT Madras)

Docker-Based Three-Machine Cyber Security Demonstration

### **Task Documentation and Implementation Report**

Technologies: Docker, Docker Compose, Python, Flask, Flask-Limiter

Environment: Private Docker Bridge Network

## ABSTRACT

This project presents a controlled cyber-security laboratory for demonstrating secure file-upload communication, abnormal request detection, alert generation, and application-layer mitigation. The system is implemented using three Docker containers connected through a private Docker bridge network. Machine 1 represents a legitimate client, Machine 2 generates controlled excessive upload traffic, and Machine 3 acts as the protected file-upload server with an integrated IDS-style monitoring mechanism and rate limiter.

The server monitors upload requests by client IP address and maintains a 60-second request window. When five upload requests are observed from a client within that window, the application generates an abnormal-activity alert. Flask-Limiter enforces a limit of five upload requests per minute per client. Requests beyond the configured limit are rejected with HTTP status 429, and the server records a mitigation message.

The final verification demonstrates that the mitigation does not stop the complete service: after excessive requests are blocked, Machine 1 can still upload a legitimate file successfully. The project is intentionally confined to a private Docker environment and is best described as a controlled DDoS/excessive-traffic simulation rather than a real Internet attack.

## 1. INTRODUCTION

Modern web applications that accept file uploads must protect their resources against excessive request rates. A client that sends repeated requests in a short period can increase CPU, memory, network, storage, and application-processing overhead. If the service does not control the request rate, legitimate users may experience reduced availability.

The objective of this project is to create a small and understandable security laboratory in which this behavior can be observed safely. Docker is used to simulate three independent machines. A legitimate client uploads a file, a second container generates controlled excessive requests, and the server detects and mitigates the abnormal request pattern.

The project combines containerization, private networking, web application development, request monitoring, IDS-style alerting, and rate limiting. It is designed for academic demonstration, project review, and viva presentation.

## 2. PROBLEM STATEMENT

A file-upload endpoint can become a target for excessive request traffic. Each upload request requires the application to parse the request, process the multipart form data, write a file, and return a response. Repeated requests can therefore consume application resources and reduce service availability.

The problem addressed by this project is how to identify an abnormal upload request rate and apply a mitigation mechanism while keeping the service available to legitimate users.

The proposed solution monitors requests to the upload endpoint, groups activity by client IP address, evaluates the number of requests within a 60-second window, generates an alert at the configured threshold, and applies rate limiting to reject subsequent excessive requests.

## 3. OBJECTIVES

- Create an isolated three-machine network using Docker containers.

- Provide a file-upload service on Machine 3.
- Allow Machine 1 to perform legitimate file uploads.
- Generate controlled excessive upload traffic from Machine 2.
- Monitor upload requests based on client IP address.
- Detect abnormal request patterns using a threshold and time window.
- Generate an IDS-style alert when the threshold is reached.
- Apply rate limiting as an application-layer mitigation.
- Return HTTP 429 responses for requests exceeding the configured limit.
- Verify that legitimate file uploads continue to work after mitigation.
- Provide a repeatable demonstration suitable for project review and viva.

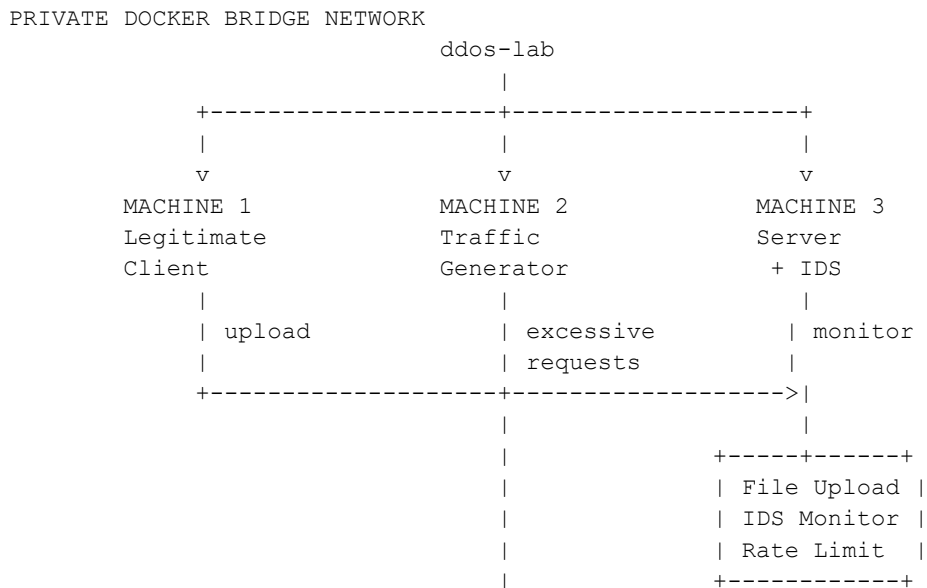
## 4. SCOPE

The scope is limited to a controlled application-layer security demonstration inside a private Docker environment. The project focuses specifically on HTTP file-upload requests and does not attempt to reproduce every characteristic of a production DDoS attack.

The current implementation does not include a distributed botnet, Internet-scale traffic, firewall-level blocking, a production network IDS, a Web Application Firewall, or a persistent SIEM. Since the demonstration uses one traffic-generator container, the technically precise description is a controlled excessive-traffic or DoS simulation with DDoS-oriented detection and mitigation concepts.

## 5. SYSTEM ARCHITECTURE

The architecture contains three Docker containers connected to one private bridge network. Each container has a clearly defined responsibility.



Machine 1 and Machine 2 access Machine 3 through the Docker service name `machine3-server`. Docker's internal DNS resolves the service name on the shared network, so the implementation does not depend on a permanent container IP address.

## 6. MACHINE ROLES

| Machine   | Role                         | Responsibility                                                                                  |
|-----------|------------------------------|-------------------------------------------------------------------------------------------------|
| Machine 1 | Legitimate client            | Creates and uploads legitimate.txt to the server.                                               |
| Machine 2 | Controlled traffic generator | Sends repeated upload requests to demonstrate excessive traffic.                                |
| Machine 3 | Server + IDS                 | Hosts the Flask upload service, monitors requests, generates alerts, and applies rate limiting. |

## 7. TECHNOLOGIES AND TOOLS

| Technology            | Purpose                                                               |
|-----------------------|-----------------------------------------------------------------------|
| Docker                | Runs isolated containers representing the three machines.             |
| Docker Compose        | Defines the services, network, build configuration, and port mapping. |
| Docker Bridge Network | Provides private communication among the three containers.            |
| Python 3.12           | Runtime for the Flask application and client containers.              |
| Flask                 | Implements the HTTP file-upload service.                              |
| Flask-Limiter         | Provides request-rate limiting.                                       |
| curl                  | Tests connectivity and performs multipart file-upload requests.       |
| PowerShell / Bash     | Used to control the Docker environment and execute tests.             |

## 8. NETWORK DESIGN

The project uses a Docker bridge network called `ddos-lab` in the Compose configuration. The Compose-created network observed during the completed implementation was `ddos-file-upload-lab_ddos-lab`. The network used the Docker bridge driver.

During testing, the observed network subnet was `172.18.0.0/16` and the gateway was `172.18.0.1`. Example container addresses during one run were Machine 1 at `172.18.0.2`, Machine 2 at `172.18.0.3`, and Machine 3 at `172.18.0.4`. These addresses can change when containers are recreated.

For that reason, the project uses `machine3-server` rather than a hard-coded IP address for internal communication. Docker's internal name resolution maps the service name to the current container address.

## 9. PROJECT DIRECTORY STRUCTURE

```
DAY_13/
├── .gitignore
├── ReadMe.md
├── docker-compose.yml
├── server/
│   └── app.py
```

```
|— Dockerfile
|— requirements.txt
```

The root directory contains the Docker Compose configuration. The server directory contains the Flask application, Dockerfile, and Python dependencies.

## 10. DOCKER COMPOSE CONFIGURATION

```
services:

  machine3:
    build: ./server
    container_name: machine3-server
    ports:
      - "8080:5000"
    networks:
      - ddos-lab

  machine1:
    image: python:3.12-slim
    container_name: machine1-client
    command: ["sleep", "infinity"]
    networks:
      - ddos-lab

  machine2:
    image: python:3.12-slim
    container_name: machine2-attacker
    command: ["sleep", "infinity"]
    networks:
      - ddos-lab

networks:
  ddos-lab:
    driver: bridge
```

Machine 3 is built from the local server directory. Machine 1 and Machine 2 use the Python 3.12 slim image and remain running using sleep infinity so that they can be entered interactively.

The ports mapping 8080:5000 publishes the Flask server's internal port 5000 as port 8080 on the Docker host. Internal clients use machine3-server:5000.

## 11. SERVER DOCKERFILE

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

RUN mkdir -p /app/uploads
```

```
EXPOSE 5000
```

```
CMD ["python", "-u", "app.py"]
```

The Dockerfile creates a lightweight Python environment, installs Flask and Flask-Limiter, copies the application, creates the upload directory, exposes port 5000, and starts the application using unbuffered Python output. Unbuffered output helps the IDS and mitigation messages appear immediately in docker logs.

## 12. PYTHON DEPENDENCIES

```
Flask  
Flask-Limiter
```

Flask provides the web framework and HTTP routing. Flask-Limiter provides the request-rate control used by the upload endpoint.

## 13. SERVER APPLICATION

Machine 3 implements a Flask application with an upload folder at /app/uploads. The / route returns a simple server identification message. The /upload endpoint accepts POST requests containing a multipart file.

```
from flask import Flask, request, jsonify  
from flask_limiter import Limiter  
from flask_limiter.util import get_remote_address  
import os  
import time  
  
app = Flask(__name__)  
  
UPLOAD_FOLDER = "/app/uploads"  
os.makedirs(UPLOAD_FOLDER, exist_ok=True)  
  
limiter = Limiter(  
    key_func=get_remote_address,  
    app=app,  
    default_limits=[]  
)
```

The `get_remote_address` function is used as the limiter key. Therefore, rate limiting is applied per client IP address in the controlled environment.

## 14. FILE-UPLOAD PROCESS

When a request reaches /upload, the application first checks whether a file field exists. It then checks whether a filename is present. If the request is valid, the file is saved into /app/uploads. The application prints an upload log and returns a JSON success response.

```
if "file" not in request.files:  
    return jsonify(  
        "status": "error",  
        "message": "No file provided"  
    ), 400  
  
file = request.files["file"]  
filename = file.filename
```

```

if not filename:
    return jsonify({
        "status": "error",
        "message": "Invalid filename"
    }), 400

filepath = os.path.join(UPLOAD_FOLDER, filename)
file.save(filepath)

```

For a production system, additional controls such as authentication, authorization, filename sanitization, file-type validation, maximum file size, malware scanning, and safe storage would be required.

## 15. IDS MONITORING MECHANISM

The IDS component is implemented as application-level request monitoring. It maintains a dictionary called `request_times`. Each client IP address has a list of timestamps representing recent upload requests.

```

request_times = {}
ALERT_THRESHOLD = 5
MONITOR_WINDOW = 60

@app.before_request
def monitor_requests():

    if request.path != "/upload":
        return

    client_ip = request.remote_addr
    current_time = time.time()

    if client_ip not in request_times:
        request_times[client_ip] = []

    request_times[client_ip] = [
        timestamp
        for timestamp in request_times[client_ip]
        if current_time - timestamp < MONITOR_WINDOW
    ]

    request_times[client_ip].append(current_time)

    request_count = len(request_times[client_ip])

    print(
        f"[MONITOR] Upload request #{request_count} "
        f"from {client_ip}",
        flush=True
    )

    if request_count == ALERT_THRESHOLD:
        print(
            f"[ALERT] Abnormal upload activity detected "
            f"from {client_ip} "
            f"({request_count} requests in 60 seconds)",
            flush=True
        )

```

The monitoring function runs before the upload route. It removes timestamps older than 60 seconds, adds the current request, calculates the number of recent requests, and prints a monitoring message. When the count reaches five, the system prints an abnormal-activity alert.

## 16. RATE LIMITING AND MITIGATION

The upload endpoint is protected using Flask-Limiter.

```
@limiter.limit("5 per minute")
def upload():
```

This rule permits up to five upload requests per minute for a client. When additional requests exceed the configured rate, Flask-Limiter produces a 429 response.

```
@app.errorhandler(429)
def rate_limit_handler(error):

    client_ip = request.remote_addr

    print(
        f"[MITIGATION] Excessive traffic blocked "
        f"from {client_ip}",
        flush=True
    )

    return jsonify({
        "status": "blocked",
        "message": "Rate limit exceeded"
    }), 429
```

HTTP 429 means Too Many Requests. In this project it is the observable evidence that the mitigation has rejected excessive upload requests.

## 17. COMPLETE REQUEST FLOW

Normal flow: Machine 1 creates legitimate.txt and sends it to Machine 3 using a multipart POST request. The IDS records the request. Because the request rate is within the allowed range, Flask-Limiter permits the request. The file is saved and HTTP 200 is returned.

Excessive-traffic flow: Machine 2 repeatedly sends upload requests. The IDS records the requests. When the fifth request occurs within the 60-second monitoring window, an IDS alert is generated. Requests beyond the rate limit are rejected with HTTP 429 and the mitigation handler logs the blocked activity.

Recovery/availability flow: Machine 1 attempts another legitimate upload after the excessive traffic has been blocked. The request succeeds, showing that the server remains available.

## 18. PROJECT SETUP

### 18.1 Check Docker

```
docker --version
docker compose version
```

The completed environment used Docker 29.8.0 and Docker Compose v5.5.1.

## 18.2 Open the project directory

```
cd C:\Users\jaasw\ddos-file-upload-lab
```

## 18.3 Validate Compose configuration

```
docker compose config
```

This validates the Compose file before starting the containers.

## 18.4 Build and start the environment

```
docker compose up -d --build
```

## 18.5 Verify containers

```
docker ps
```

The expected containers are machine1-client, machine2-attacker, and machine3-server.

# 19. TESTING AND DEMONSTRATION

## 19.1 Test server from the host

```
curl http://localhost:8080/
```

Expected response:

```
Machine 3 - Secure File Upload Server
```

## 19.2 Test legitimate Machine 1

```
docker exec -it machine1-client bash
echo "This is a legitimate file from Machine 1" > legitimate.txt
cat legitimate.txt
curl -s -F "file=@legitimate.txt" http://machine3-server:5000/upload
```

Expected response:

```
{"message":"File uploaded successfully","status":"success"}
```

## 19.3 Verify the file on Machine 3

```
docker exec -it machine3-server bash
ls -l /app/uploads
cat /app/uploads/legitimate.txt
exit
```

The file legitimate.txt was verified in /app/uploads with the expected content.

## 19.4 Start Machine 2

```
docker exec -it machine2-attacker bash
echo "Controlled traffic test from Machine 2" > attack.txt
cat attack.txt
```

## 19.5 Send controlled excessive traffic

```
for i in {1..10}; do
    echo "Request $i"
    curl -s -o /dev/null -w "HTTP Status: %{http_code}\n" -F "file=@attack.txt"
    http://machine3-server:5000/upload
done
```

The completed test produced HTTP 200 for requests 1 through 5 and HTTP 429 for subsequent requests during the rate-limit period.

## 19.6 Observe live IDS logs

```
docker logs -f machine3-server
```

The live log terminal shows monitoring, alert, upload, and mitigation messages.

## 19.7 Show the actual blocked response

```
curl -i -F "file=@attack.txt" http://machine3-server:5000/upload
```

The expected result is HTTP/1.1 429 TOO MANY REQUESTS with a JSON message indicating that the rate limit was exceeded.

## 19.8 Verify legitimate service after mitigation

```
docker exec -it machine1-client bash
curl -s -F "file=@legitimate.txt" http://machine3-server:5000/upload
```

The expected response is HTTP 200 with a successful upload message. This is a key result because it demonstrates that the mitigation blocks excessive requests while the service continues to accept legitimate traffic.

## 20. OBSERVED RESULTS

| Test                        | Result      | Meaning                                          |
|-----------------------------|-------------|--------------------------------------------------|
| Host-to-server connectivity | HTTP 200    | Machine 3 is reachable.                          |
| Machine 1 upload            | HTTP 200    | Legitimate upload is accepted.                   |
| Server file verification    | File exists | File reached and was stored by Machine 3.        |
| Machine 2 requests 1-5      | HTTP 200    | Traffic is initially within the configured rate. |
| Fifth request               | IDS alert   | Threshold condition is detected.                 |
| Machine 2 requests 6-10     | HTTP 429    | Excessive traffic is blocked.                    |
| Machine 1 after mitigation  | HTTP 200    | Legitimate service operation continues.          |

## 21. LOG AND ALERT ANALYSIS

Representative logs from the completed test were:

```
[MONITOR] Upload request #5 from 172.18.0.2
[ALERT] Abnormal upload activity detected from 172.18.0.2
(5 requests in 60 seconds)
```

```
[UPLOAD] File 'attack.txt' uploaded from 172.18.0.2
```

```
[MONITOR] Upload request #6 from 172.18.0.2
```

```
[MITIGATION] Excessive traffic blocked from 172.18.0.2
```

The log sequence demonstrates the detection and response chain. The monitoring component observes the request. The fifth request causes the alert. The rate limiter subsequently blocks excessive requests, and the mitigation handler records the event.

The application uses `flush=True` in its print statements, and the Dockerfile starts Python with `-u`. These settings help the logs appear immediately during a live demonstration.

## 22. SECURITY ANALYSIS

- Isolation: all demonstration traffic is confined to a private Docker network.
- Monitoring: upload requests are tracked by client IP and time window.
- Detection: a threshold identifies high request activity.
- Alerting: the server generates an explicit abnormal-activity alert.
- Mitigation: rate limiting rejects requests beyond the configured rate.
- Observability: HTTP status codes and logs provide visible evidence of the security response.
- Availability testing: the legitimate client is tested again after mitigation.

The security mechanism is an application-layer control. It complements, rather than replaces, production network security controls such as firewalls, reverse proxies, WAFs, network IDS/IPS, load balancers, and centralized monitoring.

## 23. LIMITATIONS

- Only one traffic-generator container is used. Therefore, this is a controlled excessive-traffic/DoS simulation rather than a full distributed botnet-based DDoS.
- Flask-Limiter uses in-memory storage in the current implementation. The server logs may warn that in-memory storage is not recommended for production.
- Request history is stored in process memory and is lost when the application restarts.
- The threshold and rate limit are fixed values rather than adaptive production detection.
- The current upload implementation should add stronger filename, file-type, file-size, authentication, authorization, and storage controls for production.
- There is no persistent security database or external alerting platform.
- The IDS logic is part of the Flask application rather than a separate network IDS sensor.
- Container IP addresses can change after recreation, so service-name addressing is preferred.

## 24. FUTURE ENHANCEMENTS

- Use Redis or another shared backend for production-grade rate-limiter state.
- Add user authentication and authorization.
- Add file size and file-type restrictions.
- Sanitize uploaded filenames and store files safely.
- Add persistent security event storage.
- Create a dashboard for request counts, alerts, and blocked requests.

- Add reverse-proxy or WAF-based controls.
- Use multiple controlled traffic-generator containers to demonstrate distributed sources safely in the laboratory.
- Make thresholds configurable through environment variables.
- Add adaptive detection based on request rate and resource utilization.
- Integrate alerts with a SIEM or monitoring platform.
- Add HTTPS/TLS and secure upload storage.
- Create automated tests for upload, validation, detection, mitigation, and recovery.

## 25. LIVE SHOWCASE SCRIPT

The recommended presentation sequence is designed to make the project easy for an evaluator to understand.

### Step 1 – Start the environment

Run `docker compose up -d --build` and show `docker ps`.

### Step 2 – Show the network

Run `docker network inspect ddos-file-upload-lab_ddos-lab` and explain that all three machines are on a private bridge network.

### Step 3 – Show Machine 3

Run `docker logs -f machine3-server` and keep this terminal visible.

### Step 4 – Show normal operation

Enter Machine 1, display `legitimate.txt`, and upload it. Show the HTTP 200 response and server log.

### Step 5 – Verify storage

Show `/app/uploads/legitimate.txt` on Machine 3.

### Step 6 – Start excessive traffic

Enter Machine 2 and show `attack.txt`.

### Step 7 – Generate controlled traffic

Run the ten-request loop. Explain that this is confined to the private lab.

### Step 8 – Show detection

Point to the [ALERT] message at five requests in 60 seconds.

### Step 9 – Show mitigation

Point to HTTP 429 and the [MITIGATION] messages for excessive requests.

### Step 10 – Prove availability

Return to Machine 1 and upload `legitimate.txt` again. Show HTTP 200.

## 26. VIVA QUESTIONS AND ANSWERS

### What is the purpose of this project?

It demonstrates secure file-upload communication and controlled excessive-traffic detection and mitigation using Docker.

### Why are three machines used?

They separate the legitimate client, traffic generator, and protected server so the security flow can be demonstrated clearly.

### Why is Docker used?

Docker provides isolated and reproducible environments and allows the machines to communicate through a controlled private network.

### Why is Docker Compose used?

Compose defines the three services, their images, networking, and server port mapping in one configuration.

### What does the IDS monitor?

It monitors upload requests to /upload and tracks request timestamps for each client IP.

### What is the detection threshold?

Five upload requests within a 60-second monitoring window.

### What happens after the rate limit is exceeded?

Flask-Limiter rejects additional requests and the application returns HTTP 429.

### What does HTTP 429 mean?

Too Many Requests. It indicates that the configured request rate has been exceeded.

### Does mitigation stop the entire server?

No. The final test shows that Machine 1 can still upload successfully after excessive requests are blocked.

### Is this a real Internet DDoS attack?

No. It is a controlled DDoS/excessive-traffic simulation performed inside a private Docker laboratory.

### Why use machine3-server instead of an IP address?

Docker DNS resolves the service name, while container IP addresses can change after recreation.

### What is the difference between detection and mitigation?

Detection identifies the abnormal pattern and generates the alert. Mitigation rejects subsequent excessive requests.

## 27. IMPORTANT COMMAND REFERENCE

| Command                       | Purpose               |
|-------------------------------|-----------------------|
| <code>docker --version</code> | Check Docker version. |

|                                                      |                                   |
|------------------------------------------------------|-----------------------------------|
| docker compose version                               | Check Compose version.            |
| docker compose config                                | Validate Compose configuration.   |
| docker compose up -d --build                         | Build and start all containers.   |
| docker ps                                            | Show running containers.          |
| docker ps -a                                         | Show all containers.              |
| docker exec -it machine1-client bash                 | Enter Machine 1.                  |
| docker exec -it machine2-attacker bash               | Enter Machine 2.                  |
| docker exec -it machine3-server bash                 | Enter Machine 3.                  |
| docker logs -f machine3-server                       | Follow live server logs.          |
| docker network inspect ddos-file-upload-lab_ddos-lab | Inspect the private network.      |
| curl http://localhost:8080/                          | Test host-to-server connectivity. |
| docker compose down                                  | Stop and remove the lab.          |

## 28. CONCLUSION

The project successfully implements a controlled three-machine Docker security laboratory for demonstrating secure file upload, abnormal request detection, alert generation, and application-layer mitigation. Machine 1 acts as a legitimate client, Machine 2 generates controlled excessive traffic, and Machine 3 acts as the protected server with IDS-style monitoring and rate limiting.

The completed testing demonstrated the complete security cycle. Normal file uploads were accepted. The fifth upload request within the monitoring window generated an abnormal-activity alert. Requests beyond the configured rate were rejected with HTTP 429. The server logs clearly displayed mitigation events. Finally, a legitimate upload from Machine 1 succeeded after the excessive traffic was blocked.

The project therefore demonstrates the practical relationship between monitoring, detection, alerting, mitigation, and availability verification. Because the environment is isolated inside Docker, the experiment is repeatable and suitable for academic demonstration. For production use, the design would need persistent storage, stronger upload validation, authentication, distributed rate-limiter state, external monitoring, and additional network security controls.

## APPENDIX A – COMPLETE CONTROLLED TRAFFIC TEST

```
for i in {1..10}; do
    echo "Request $i"
    curl -s -o /dev/null -w "HTTP Status: %{http_code}\n" -F "file=@attack.txt"
    http://machine3-server:5000/upload
done
```